Testing philosophy

Methodology Summary:

1. **Risk-based testing:** Focused effort on areas with the highest risk of failure and business impact, such as data integrity in student registration. Added focus on data integrity risks in student-course relationships and user interface consistency issues that affect user experience.
2. **Code-level validation:** Conducted line-by-line analysis to identify exact locations of security vulnerabilities, architectural violations, and quality issues.
3. **Requirements traceability:** Ensured every functional requirement is covered by at least one test case via a Traceability Matrix.
4. **Early feedback:** Conducted a static code review to identify bugs and quality issues before dynamic testing.
5. **Clear communication:** Documented bugs with precise steps to reproduce, expected/actual results, and visual evidence to aid developer resolution.

Testing and strategy tools

Tool Category	Tool Selected	Purpose	Justification
Test management	Excel/Google sheets	To create test plan, cases, traceability matrix, and bug reports	Accessible universally, simple, and the documentation requirements of assessment
Test automation	Python unittest	To create a harness for automated regression testing of core functions	Native to the application's tech stack; allows for CI/CD integration
Continuous Integration/Continuous Deployment integration	GitHub actions	To automate test execution on code commits	Demonstrates a modern, repeatable, and scalable testing framework as required

Top 10 identified problems

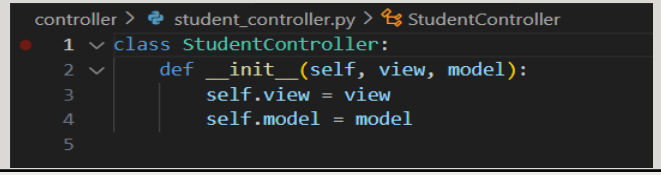
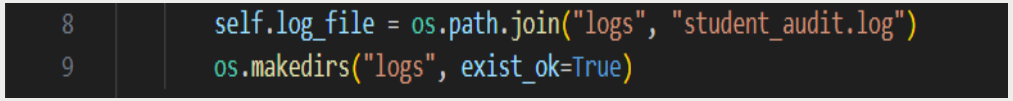
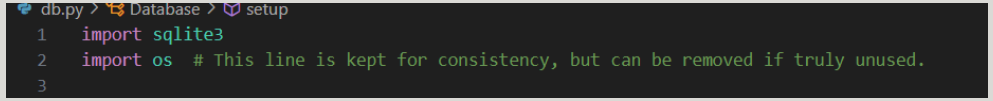
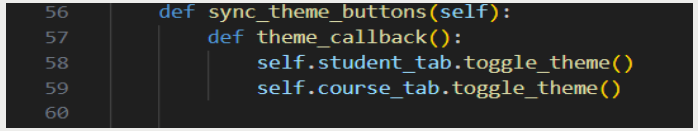
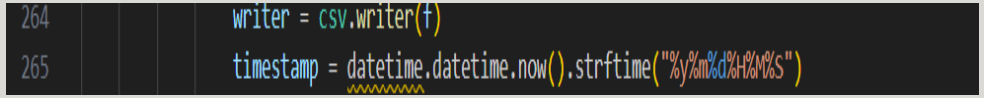
Rank	Problem	Category		Intensity
1	Lack of input validation allows SQL injection in search fields.	Security		Critical
2	UI inconsistency: student_view combines first & last name, course_view does not	Usability		High

3	Silent failure: deleting a course with enrolled students corrupts data integrity & distorts student records
	In course_model.py:
	`def delete_course(self, course_id):`
	`  query = "DELETE FROM courses WHERE id = ?"`
	`  self.db.execute(query, (course_id,))`
		Data Integrity		High

4	Multiple files: Table setup redundancy. Setup logic is repeated in db.py and model files
	In db.py:
	`def setup(self):`
	`  self.cursor.execute("""`
	`  CREATE TABLE IF NOT EXISTS courses (`
	`  id INTEGER PRIMARY KEY AUTOINCREMENT,`
	In course_model.py:
	`def create_table(self):`
	`  """Create courses table if it does not exist"""`
	`  query = """`
	`  CREATE TABLE IF NOT EXISTS courses (`
	`  id INTEGER PRIMARY KEY AUTOINCREMENT,`
		Maintainability		High

5	Error handling: generic except blocks hide the root cause of errors	Reliability		High

Top 10 identified problems continued…

Rank	Problem	Category		Intensity
6	Controller classes are empty, violating the intended model view controller architecture In student_controller.py: 	Architecture		Medium
7	Hard-coded file paths are inflexible(e.g. in student_model). In student_model.py: 	Maintainability		Medium
8	Unused imports clutter the codebase. In db.py: 	Code quality		Low
9	GUI glitch: theme button text does not update correctly on app start. In main.py: 	GUI		Low
10	Missing datetime import in student_view.py causes crash on export In student_view.py: 	Functionality		Low

Recommendations for mitigation

Issue	Recommended solution	To be carried out by	Role description
SQL injection risk	Use parameterized queries exclusively	Developer & security engineer	Implements secure coding practices and database query security
UI inconsistency	Refactor student_view to display separate first/last name columns like course_view.	UI designer & developer	UI Designer ensures visual consistency and user experience
Course deletion	Add foreign key constraints & proper deletion handling or prevent deletion of enrolled courses.	Backend developer & database developer	Implements database schema changes and referential integrity rules
Code duplication(multiple files)	Centralize table creation logic in db.py and remove create table from model classes.	Backend developer	Refactors code architecture and eliminates redundancy
Poor error handling	Replace generic exceptions with specific exception handling and user-friendly messages.	Developer	Implements proper error handling and user-friendly messaging
Empty controllers	Move business logic from views into controller classes to adhere to Model View Controller pattern.	Developer & software architect	Restructures application architecture to follow MVC pattern
Hard-coded paths	Use configurable paths and centralize configuration for all file/directory paths.	Developer	Implements configuration management & path handling
Unused imports	Remove unused imports throughout codebase.	Developer & code quality analyst	Uses static analysis tools to identify and verify removal of all unused imports
Theme glitch	Adding an initial sync method to speed up the process and response time.	Frontend developer & performance engineer	Implements theme synchronization in main.py and base_view.py for consistent initial state
Missing datetime	Add missing import at top of file.	Developer	Adds import datetime to student_view.py and validates export functionality

Implementation timeline

Phase 1 (Critical - Week 1) Fix SQL injection vulnerabilities Implement proper foreign key constraints Add specific error handling	Phase 2 (High - Week 2) Remove code duplication in table creation Standardize UI consistency Fix missing imports and file path issues
Phase 3 (Medium - Week 3) Implement proper MVC with controller logic Add comprehensive input validation Improve theme synchronization	Phase 4 (Low - Week 4) Remove unused imports Code cleanup and documentation Final testing and validation